

PySpark Interview Prep

Ordered Q&A with explanations: Spark concepts, PySpark coding, SQL, and file handling

Contents

PART 1: Spark Concepts

1. map() vs flatMap()
2. Small File Problem and coalesce()
3. Tungsten Engine
4. Driver OOM vs Executor OOM
5. Simple spark-submit command

PART 2: PySpark Coding

6. Word Count (PySpark and SQL)
7. Convert all 50 columns to StringType
8. Replace int IDs with description values
9. Write a UDF
10. Remove duplicates without window / dropDuplicates
11. Join 0001 with 1 without cast or substring

PART 3: SQL Questions

12. Employee earning more than manager
13. Second highest salary
14. Third highest salary
15. Deduplicate keeping latest record
16. Delete records older than 30 days
17. Three consecutive absences
18. Cumulative sum at month end
19. Missing monthly transactions
20. Monthly sales report incl. empty months
21. CDC logic using SQL

PART 4: File Handling and Data Quality

22. List files containing a word
23. Print files, sub-files, directories with a word
24. Process a specific file
25. Double delimiter handling
26. Identify and remove malformed records

PART 1: Spark Concepts

1. map() vs flatMap()

- **map()** applies a function to each element and returns exactly **one output per input** (1-to-1).
- **flatMap()** applies a function that returns a **collection** per input, then **flattens** the results (1-to-many, or zero).

```
rdd = sc.parallelize(["hello world", "spark is fast"])

rdd.map(lambda x: x.split(" ")).collect()
# [['hello','world'], ['spark','is','fast']] -> list of lists

rdd.flatMap(lambda x: x.split(" ")).collect()
# ['hello','world','spark','is','fast'] -> flat list
```

Say in the interview: Use flatMap when each row can produce multiple rows, as in word count. The DataFrame equivalent is `explode()`.

2. Small File Problem and how coalesce() helps

Problem: Thousands of tiny files (KBs or a few MBs) instead of fewer files near the block size (128-256 MB).

Why it hurts:

- **Metadata overhead:** the HDFS NameNode (or S3 listing) holds metadata for every file.
- **Too many tasks:** Spark creates roughly one task per file or split, so scheduling overhead exceeds actual work.
- **Slow reads:** opening and closing many files is expensive.

Common causes: high `spark.sql.shuffle.partitions` (200 by default) writing out 200 tiny files, over-partitioned writes (`partitionBy` on a high-cardinality column), and streaming micro-batches.

How coalesce() helps: it **reduces the number of partitions without a full shuffle** by merging existing partitions (narrow transformation), so fewer, larger files get written.

```
df.coalesce(10).write.mode("overwrite").parquet("/out/path")
```

	coalesce(n)	repartition(n)
Shuffle	No (merges locally)	Yes (full shuffle)
Can increase partitions	No	Yes
Partition sizes	Can be uneven	Evenly balanced
Cost	Cheap	More expensive

Other fixes: `maxRecordsPerFile`, compaction jobs, Delta Lake `OPTIMIZE`, and Adaptive Query Execution (AQE) partition coalescing. Caveat: `coalesce(1)` before a heavy transformation can bottleneck the whole stage on a single task.

3. Tungsten Engine

Project Tungsten is Spark's execution engine optimization (since Spark 1.5/2.x) that pushes performance toward bare-metal hardware limits. It has three pillars:

- **Off-heap memory management and binary format:** data is stored in a compact binary format (`UnsafeRow`) instead of JVM objects. This means less garbage collection and a smaller memory footprint.
- **Cache-aware computation:** data structures and algorithms (sort, hash) are laid out to make good use of CPU L1/L2/L3 caches.

- **Whole-stage code generation:** Spark fuses multiple operators (filter, project, and so on) into **one generated Java function**. This removes virtual function calls and keeps data in CPU registers.

Say in the interview: Tungsten works with the Catalyst optimizer. Catalyst produces the optimized plan, and Tungsten executes it efficiently. You can see whole-stage codegen in `explain()` as `*(1)` markers.

4. Driver OOM vs Executor OOM (via Spark UI)

	Driver OOM	Executor OOM
Where to look	Executors tab, "driver" row; driver logs	Stages tab, task failures; Executors tab
Symptoms	Job fails or hangs at action; <code>java.lang.OutOfMemoryError: Java heap space</code> in the driver log; app crashes before tasks finish	<code>ExecutorLostFailure</code> , "Container killed by YARN for exceeding memory limits", exit code 137 or 143; tasks retry
Typical causes	<code>collect()</code> , <code>toPandas()</code> on big data; broadcasting a huge table; too many partitions (task metadata); large <code>maxResultSize</code> results	Data skew, big partitions, huge shuffles, heavy UDFs, too many cores per executor
Spark UI evidence	Driver memory near max, high GC time	Stage task metrics: Max >> Median (skew); large Spill (Memory/Disk) ; high GC Time (red)
Fix	Avoid collect, raise <code>spark.driver.memory</code> , reduce broadcast threshold	Fix skew (salting, AQE), more partitions, raise <code>executor.memory</code> and <code>memoryOverhead</code> , fewer cores per executor

Say in the interview: Start in the Stages tab. If a few tasks are far slower or bigger than the rest, it's skew (executor side). If the application dies at an action that pulls data to the driver, it's the driver.

5. Simple spark-submit command

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --name my_etl_job \
  --driver-memory 4g \
  --executor-memory 8g \
  --executor-cores 4 \
  --num-executors 10 \
  --conf spark.sql.shuffle.partitions=200 \
  --py-files deps.zip \
  my_job.py 2026-10-07 /data/input
```

Explain each flag briefly. In `my_job.py`, read the arguments with `sys.argv[1]`, `sys.argv[2]`.

PART 2: PySpark Coding

6. Word Count (PySpark and SQL)

RDD:

```
rdd = spark.sparkContext.textFile("input.txt")
counts = (rdd.flatMap(lambda l: l.lower().split())
          .map(lambda w: (w, 1))
          .reduceByKey(lambda a, b: a + b))
```

DataFrame:

```
from pyspark.sql import functions as F
df = spark.read.text("input.txt")
(df.select(F.explode(F.split(F.lower("value"), r"\s+")).alias("word"))
 .filter("word != ''")
 .groupBy("word").count()
 .orderBy(F.desc("count")).show())
```

SQL:

```
SELECT word, COUNT(*) AS cnt
FROM (
  SELECT explode(split(lower(value), '\s+')) AS word FROM lines
) t
WHERE word <> ''
GROUP BY word
ORDER BY cnt DESC;
```

7. Convert all 50 columns to StringType

```
from pyspark.sql import functions as F
df_str = df.select([F.col(c).cast("string").alias(c) for c in df.columns])
```

Using a single `select` is better than a `withColumn` loop. Looping 50 times builds a very deep logical plan and is slower.

8. Replace int IDs (dep, team, sub_team...) with their description

Assumption: `table_1` has ~100 columns, and a lookup table `lookup(id, desc)` maps IDs to descriptions.

```
from pyspark.sql import functions as F

orig_cols = table_1.columns
cols_to_map = ["dep", "team", "sub_team"]      # extend as needed

df = table_1
for c in cols_to_map:
    lk = lookup.select(F.col("id").alias(c), F.col("desc").alias(f"{c}_desc"))
    df = (df.join(F.broadcast(lk), on=c, how="left")
          .drop(c)
          .withColumnRenamed(f"{c}_desc", c))

df = df.select(orig_cols)                      # restore original column order
```

Points to mention:

- Broadcast the lookup because it is small, which avoids shuffling the 100-column table.
- Use a **left** join so unmatched IDs aren't lost.
- If the lookup is tiny, an alternative with no joins is `create_map`:

```
from itertools import chain
m = F.create_map([F.lit(x) for x in chain(*[(r.id, r.desc) for r in lookup.collect()])])
```

```
df = df.withColumn("dep", m[F.col("dep")])
```

9. Write a UDF

```
from pyspark.sql import functions as F
from pyspark.sql.types import StringType

def grade(score):
    if score is None: return None
    return "A" if score >= 90 else "B" if score >= 75 else "C"

grade_udf = F.udf(grade, StringType())
df = df.withColumn("grade", grade_udf("score"))

# register for SQL
spark.udf.register("grade_sql", grade, StringType())
```

Say in the interview: UDFs are a black box to Catalyst, so there is no optimization, and they incur Python-JVM serialization cost. Prefer built-in functions (**when/otherwise** here), or **pandas_udf** (vectorized, Arrow-based) if a UDF is truly needed.

10. Eliminate duplicates without window functions or dropDuplicates()

Option A: groupBy (keep one row per key)

```
df.groupBy("id").agg(*[F.first(c).alias(c) for c in df.columns if c != "id"])
```

Option B: exact duplicates with SQL DISTINCT or UNION

```
SELECT DISTINCT * FROM t;
-- or
SELECT * FROM t UNION SELECT * FROM t;  -- UNION removes duplicates
```

Option C: keep the latest record using a self-join on MAX

```
SELECT t.*
FROM t
JOIN (SELECT id, MAX(updated_at) AS mx FROM t GROUP BY id) m
ON t.id = m.id AND t.updated_at = m.mx;
```

11. Join 0001 with 1 without casting or substring

Normalize the key by stripping leading zeros with a **regex** (neither cast nor substring):

```
from pyspark.sql import functions as F

a = a.withColumn("join_key", F.regexp_replace("id", r"^0+(?!$)", ""))
b = b.withColumn("join_key", F.regexp_replace("id", r"^0+(?!$)", ""))

result = a.join(b, on="join_key", how="inner")

SELECT *
FROM a JOIN b
ON regexp_replace(a.id, '^0+(?!$)', '') = regexp_replace(b.id, '^0+(?!$)', '');
```

The **(?!\$)** keeps a lone "0" from becoming an empty string. An alternative is **ltrim('0' FROM id)** or **lpad(id, 4, '0')** on both sides, if the max length is known. Normalizing the key before the join is cleaner than putting expressions in the **ON** clause.

PART 3: SQL Questions

12. Employee earning more than their manager

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS mgr_salary
FROM employee e
JOIN employee m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

A self-join treats the same table as both employee and manager.

13. Second highest salary

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

Or with a window function (handles ties properly):

```
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rn FROM employee
) t WHERE rn = 2 LIMIT 1;
```

14. Third highest salary

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rn FROM employee
) t WHERE rn = 3;
```

Say in the interview: The Nth highest is just `rnk = N`. Use **DENSE_RANK** (not **ROW_NUMBER** or **RANK**) so duplicate salaries don't skip ranks.

15. Deduplicate and keep the latest record

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) rn
FROM t
) x WHERE rn = 1;
```

```
from pyspark.sql.window import Window
w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

16. Delete records older than 30 days

```
DELETE FROM transactions
WHERE txn_date < date_sub(current_date(), 30);
```

(PostgreSQL/MySQL: `WHERE txn_date < CURRENT_DATE - INTERVAL '30 days'`.)

Note: Plain Parquet/Hive tables don't support row-level **DELETE**. It works on **Delta/Iceberg/Hudi**. For plain files, filter and overwrite:

```
df.filter(F.col("txn_date") >= F.date_sub(F.current_date(), 30)) \
  .write.mode("overwrite").parquet("/new/path") # write to a new path, then swap
```

17. Three consecutive absences (gaps-and-islands)

```
WITH absent AS (
  SELECT student_id, att_date,
         ROW_NUMBER() OVER (PARTITION BY student_id ORDER BY att_date) AS rn
FROM attendance
WHERE status = 'Absent'
```

```

)
SELECT student_id,
       MIN(att_date) AS start_date,
       MAX(att_date) AS end_date,
       COUNT(*)      AS days_absent
FROM absent
GROUP BY student_id, date_sub(att_date, rn)
HAVING COUNT(*) >= 3;

```

Logic: For consecutive dates, `date - row_number` is constant, so each streak forms one group. Count each group. If weekends or holidays have no rows, the "consecutive" definition needs a calendar table.

18. Cumulative sum as of the last date of every month

```

WITH monthly AS (
  SELECT date_trunc('month', txn_date) AS m, SUM(amount) AS amt
  FROM transactions
  GROUP BY date_trunc('month', txn_date)
)
SELECT last_day(m) AS month_end,
       amt AS month_total,
       SUM(amt) OVER (ORDER BY m) AS cumulative_sum
FROM monthly
ORDER BY m;

```

Aggregate to the monthly level first, then take a running sum, which gives the cumulative value at each month's end.

19. What if there is no transaction in a month?

The query above **skips** the empty month. Fix it by generating a **calendar of months** and left-joining:

```

WITH monthly AS (
  SELECT date_trunc('month', txn_date) AS m, SUM(amount) AS amt
  FROM transactions GROUP BY 1
),
bounds AS (SELECT MIN(m) mn, MAX(m) mx FROM monthly),
cal AS (
  SELECT explode(sequence(mn, mx, interval 1 month)) AS m FROM bounds
)
SELECT last_day(c.m) AS month_end,
       COALESCE(mo.amt, 0) AS month_total,
       SUM(COALESCE(mo.amt, 0)) OVER (ORDER BY c.m) AS cumulative_sum
FROM cal c
LEFT JOIN monthly mo ON c.m = mo.m
ORDER BY c.m;

```

An empty month gets 0 for the month total, and the cumulative sum **carries forward** automatically.

20. Monthly sales report (Jan, Feb, ... including months with no sales)

```

WITH months AS (
  SELECT explode(sequence(1, 12)) AS month_num
)
SELECT m.month_num,
       date_format(make_date(2026, m.month_num, 1), 'MMMM') AS month_name,
       COALESCE(SUM(s.amount), 0) AS total_sales
FROM months m
LEFT JOIN sales s
  ON month(s.sale_date) = m.month_num
 AND year(s.sale_date) = 2026
GROUP BY m.month_num
ORDER BY m.month_num;

```

Note the year filter goes in the **ON** clause, not **WHERE**. A **WHERE** filter would turn the left join into an inner join and drop the empty months again.

PySpark pivot version:

```
(df.groupBy(F.year("sale_date").alias("yr"))
  .pivot("month", list(range(1, 13)))
  .sum("amount").fillna(0))
```

21. CDC (Change Data Capture) using SQL

Compare the **source (new snapshot)** with the **target (existing)** on the business key:

```
SELECT COALESCE(s.id, t.id) AS id,
CASE
  WHEN t.id IS NULL THEN 'INSERT'
  WHEN s.id IS NULL THEN 'DELETE'
  WHEN md5(concat_ws('|', s.name, s.salary, s.dept))
    <> md5(concat_ws('|', t.name, t.salary, t.dept)) THEN 'UPDATE'
  ELSE 'NO_CHANGE'
END AS change_type
FROM source s
FULL OUTER JOIN target t ON s.id = t.id;
```

Applying the changes with **MERGE** (Delta/Iceberg):

```
MERGE INTO target t
USING source s ON t.id = s.id
WHEN MATCHED AND (t.name <> s.name OR t.salary <> s.salary) THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

The hash comparison avoids writing out every column comparison. For history tracking, extend this to **SCD Type 2** with **start_date**, **end_date**, and **is_current**.

PART 4: File Handling and Data Quality

22. List all files containing a specific word

Inside the file content:

```
grep -rl "word" /path/to/dir      # -r recursive, -l list names only
grep -rli "word" /path/to/dir    # case-insensitive
```

In the file name:

```
find /path/to/dir -type f -name "*word"
```

Python:

```
import os
for root, dirs, files in os.walk("/path"):
    for f in files:
        p = os.path.join(root, f)
        try:
            if "word" in open(p, errors="ignore").read():
                print(p)
        except Exception:
            pass
```

23. Print files, sub-files, and directories containing a specific word

By name (files and folders):

```
find /path -iname "*word*"      # files and directories
find /path -type d -iname "*word*" # directories only
find /path -type f -iname "*word*" # files only
```

By content (show line matches too):

```
grep -rn "word" /path          # file:line:text

for root, dirs, files in os.walk("/path"):
    for d in dirs:
        if "word" in d.lower(): print("DIR :", os.path.join(root, d))
    for f in files:
        if "word" in f.lower(): print("FILE:", os.path.join(root, f))
```

On HDFS: `hdfs dfs -ls -R /path | grep word`.

24. Process a specific file from a directory

```
# exact file
df = spark.read.option("header", True).csv("/data/input/sales_2026-10-07.csv")

# pattern/glob
df = spark.read.option("header", True).csv("/data/input/sales_2026-10-*.csv")

# file name passed as a spark-submit argument
import sys
df = spark.read.csv(sys.argv[1], header=True)

# know which file each row came from
df.withColumn("src_file", F.input_file_name())
```

For listing and choosing the latest file, use `hdfs dfs -ls` or `dbutils.fs.ls` and pick by name or timestamp.

25. Handling double delimiters in input files

Example record: `101||John||Sales`

Spark 3.x+ supports multi-character delimiters:

```
df = spark.read.option("sep", "|").option("header", True).csv("file.txt")
```

Fallback for older versions (read as text and split with a regex):

```
raw = spark.read.text("file.txt")
parts = F.split("value", r"\\|", -1) # -1 keeps trailing empty fields
df = raw.select(parts[0].alias("id"), parts[1].alias("name"), parts[2].alias("dept"))
```

Say in the interview: In `split`, `|` is a regex special character, so it must be escaped. Use limit `-1` so empty trailing values aren't dropped. Also check for delimiters inside data, which need quote or escape options.

26. Identify and remove malformed/bad records

Spark CSV and JSON readers have three parse modes:

Mode	Behavior
PERMISSIVE (default)	Keeps bad rows, sets fields to null, and stores the raw line in a corrupt-record column
DROPMALFORMED	Silently drops bad rows
FAILFAST	Throws an exception on the first bad row

Best practice: capture the bad records instead of silently dropping them.

```
from pyspark.sql.types import *

schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("salary", DoubleType()),
    StructField("_corrupt_record", StringType())
])

df = (spark.read.schema(schema)
     .option("mode", "PERMISSIVE")
     .option("columnNameOfCorruptRecord", "_corrupt_record")
     .csv("input.csv"))

df.cache() # required: Spark disallows queries on only the corrupt column from a raw file
bad = df.filter("_corrupt_record IS NOT NULL")
good = df.filter("_corrupt_record IS NULL").drop("_corrupt_record")

bad.write.mode("overwrite").csv("/quarantine/bad_records")
```

Also mention `badRecordsPath` on Databricks, and validation rules such as null checks, type checks, and column-count checks.